Programación Orientada a Objetos en Java

Análisis Comparativo y Estructural: Clases Abstractas vs. Interfaces

1. Tabla de Comparación Ampliada

Característica	Clase Abstracta (`abstract class`)	Interfaz (`interface`)
Definición y Propósito	Modela una jerarquía jerárquica de herencia compartiendo estado ("es un").	Define un contrato de capacidades o comportamientos independientes ("puede hacer").
Herencia múltiple	No permitida. Una clase solo puede extender (`extends`) de una única clase abstracta.	Permitida. Una clase o interfaz puede implementar/extender múltiples interfaces (`implements`).
Variables / Atributos	Puede tener atributos de instancia de cualquier modificador de acceso (`private` , `protected` , etc.).	Solo constantes públicas implícitas: `public static final` .
Constructores	Sí posee constructores (invocados mediante `super()` desde las subclases).	No posee constructores (no se pueden instanciar directamente ni inicializar por estado).
Métodos admitidos	Métodos abstractos (sin cuerpo) y métodos concretos (con implementación completa).	Abstractos, además de métodos `default` y `static` (desde Java 8+).
Velocidad de ejecución	Ligeramente más rápida en acceso directo a variables de instancia.	Involucra un leve sobrecosto de resolución de tabla de métodos virtuales múltiples.
Evolución de API	Permite añadir nuevos métodos con implementaciones por defecto con menor fricción en subclases.	Ideal para añadir comportamientos transversales sin romper la jerarquía base.

2. Diagrama de Clases UML del Código Fuente

El siguiente diagrama representa la arquitectura estructurada a partir del paquete `Clase_abstracta_e_interfaz`. Muestra la jerarquía de herencia con la clase abstracta `Figura`, la implementación del contrato de la interfaz `Imprimible`, y las clases concretas `Circulo` y `Rectangulo`.



Nota: La clase `Circulo` hereda de `Figura` (relación *es un*) y simultáneamente implementa el contrato `Imprimible` (relación *puede hacer*).